

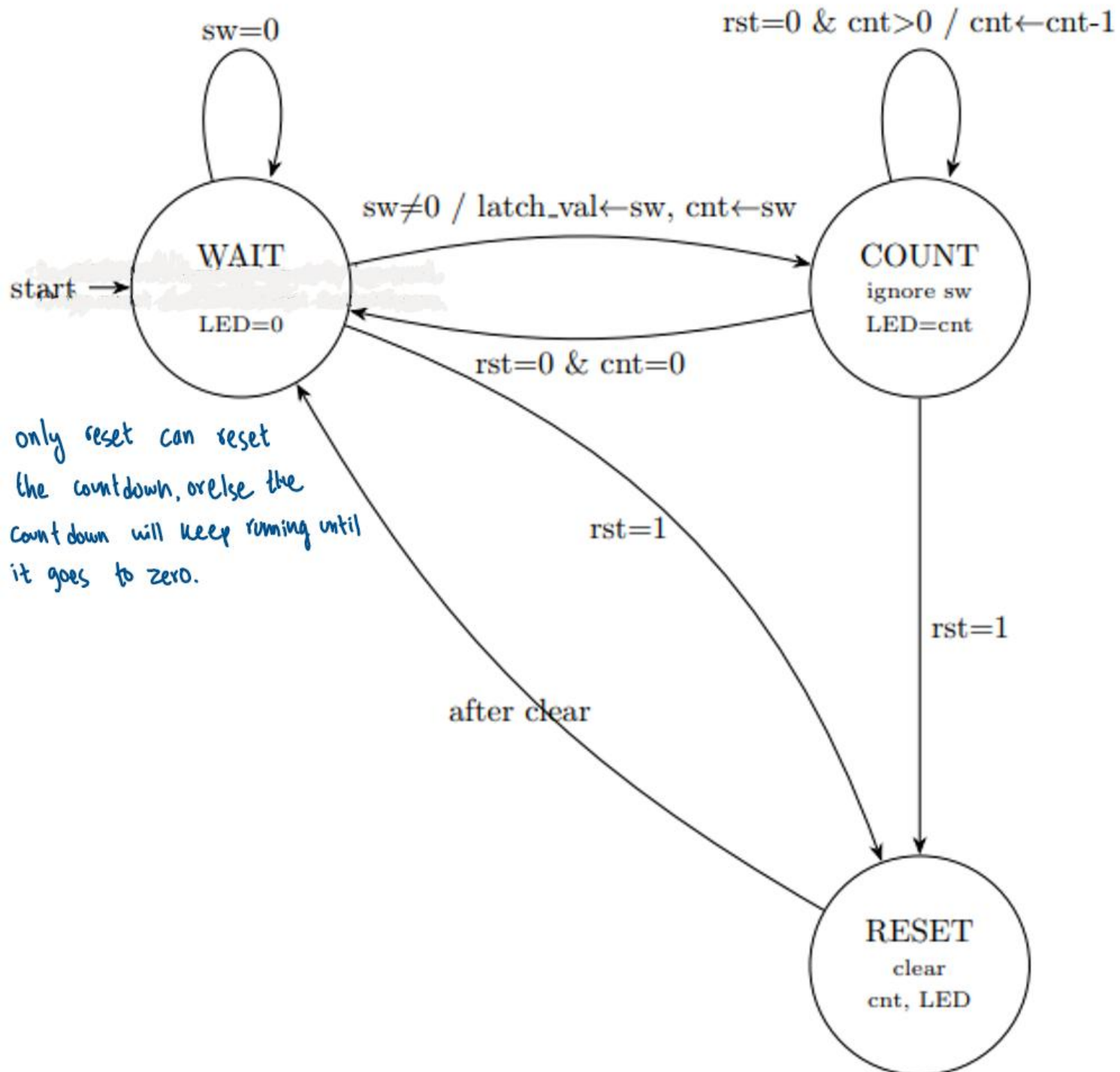


Syed Sualah Abbas
Syed Zaid Naqvi
Syed Mohsin

Lab 5: Designing FSM Using FPGA Switches and LEDs

Task 1:

FSM Design and State Diagram



Task 2: Verilog FSM and Counter Implementation

Verilog source file for FSM and counter

```
`timescale 1ns / 1ps
module debouncer(
    input wire clk,
    input wire pbin,
    output reg pbout
);

    // Debounce using a counter: output changes only when input stays stable long enough
    // With 100 MHz, COUNT_MAX=2_000_000 ~ 20ms
    parameter integer COUNT_MAX = 2_000_000;

    reg sync1 = 0, sync2 = 0;
    reg [31:0] cnt = 0;
    reg stable = 0;

    always @(posedge clk) begin
        // 2-flop synchronizer
        sync1 <= pbin;
        sync2 <= sync1;

        if (sync2 == stable) begin
            cnt <= 0; // already stable
        end else begin
            if (cnt == COUNT_MAX-1) begin
                stable <= sync2;
                cnt <= 0;
            end else begin
                cnt <= cnt + 1;
            end
        end

        pbout <= stable;
    end

endmodule

`timescale 1ns / 1ps
module switches(
    input wire clk, rst,
    input wire [15:0] btns, // not used here
    input wire [31:0] writeData, // not used
    input wire writeEnable, // not used
    input wire readEnable,
    input wire [29:0] memAddress,
    input wire [15:0] switches,

    output reg [31:0] readData
);

    reg [7:0] switchData [3:0];

    // Address for switches peripheral
    localparam [29:0] SW_ADDR = 30'h00000001;

    always @(posedge clk) begin
        if (rst) begin
            readData <= 32'd0;
            switchData[0] <= 8'd0;
            switchData[1] <= 8'd0;
            switchData[2] <= 8'd0;
        end
    end
endmodule
```



```
        switchData[3] <= 8'd0;
    end else begin
        // store switch bytes (optional, matches your skeleton)
        switchData[0] <= switches[7:0];
        switchData[1] <= switches[15:8];
        switchData[2] <= 8'd0;
        switchData[3] <= 8'd0;

        // read transaction
        if (readEnable && memAddress == SW_ADDR)
            readData <= {16'd0, switches};
        else
            readData <= 32'd0;
        end
    end
end

endmodule

`timescale 1ns / 1ps
module leds(
    input wire    clk,
    input wire    rst,
    input wire [31:0] writeData,
    input wire    writeEnable,
    input wire    readEnable, // not used
    input wire [29:0] memAddress,

    output reg [31:0] readData = 0, // not to be read
    output reg [15:0] leds
);

    reg [7:0] ledData [3:0];

    // Address for LED peripheral
    localparam [29:0] LED_ADDR = 30'h00000002;

    integer i;
    always @(posedge clk) begin
        if (rst) begin
            leds <= 16'd0;
            for (i = 0; i < 4; i = i + 1)
                ledData[i] <= 8'd0;
        end else begin
            if (writeEnable && memAddress == LED_ADDR) begin
                ledData[0] <= writeData[7:0];
                ledData[1] <= writeData[15:8];
                ledData[2] <= writeData[23:16];
                ledData[3] <= writeData[31:24];

                leds <= writeData[15:0];
            end
        end
    end
end

endmodule

`timescale 1ns / 1ps

module Task01_using_peripherals #(
    parameter integer DIV_MAX = 100_000_000
)(
    input wire    clk,
    input wire    btnC,    // RESET raw
    input wire [15:0] sw,    // physical switches
    output wire [6:0] seg,
```



```
output wire [3:0] an,
output wire [15:0] led // driven by leds module
);

// -----
// Debounced reset
// -----
wire rst;
debouncer u_db (
    .clk(clk),
    .pbin(btnC),
    .pbout(rst)
);

// -----
// MMIO addressing
// -----
localparam [29:0] SW_ADDR = 30'h00000001;
localparam [29:0] LED_ADDR = 30'h00000002;

// Switch peripheral signals
reg    sw_readEnable;
wire [31:0] sw_readData;

// LED peripheral signals
reg [31:0] led_writeData;
reg    led_writeEnable;

// Instantiate peripherals
switches u_switches (
    .clk(clk),
    .rst(rst),
    .btns(16'd0),
    .writeData(32'd0),
    .writeEnable(1'b0),
    .readEnable(sw_readEnable),
    .memAddress(SW_ADDR),
    .switches(sw),
    .readData(sw_readData)
);

leds u_leds (
    .clk(clk),
    .rst(rst),
    .writeData(led_writeData),
    .writeEnable(led_writeEnable),
    .readEnable(1'b0),
    .memAddress(LED_ADDR),
    .readData(),
    .leds(led)
);

// -----
// 1 Hz tick generator
// -----
reg [31:0] div;
wire tick_1hz = (div == (DIV_MAX-1));

always @(posedge clk) begin
    if (rst)
        div <= 0;
    else if (tick_1hz)
        div <= 0;
end
```

```
    else
        div <= div + 1;
    end

// -----
// Priority encoder
// -----
function [3:0] encode;
    input [15:0] s;
    begin
        casex (s)
            16'b1xxx_xxxx_xxxx_xxxx: encode = 4'd15;
            16'b01xx_xxxx_xxxx_xxxx: encode = 4'd14;
            16'b001x_xxxx_xxxx_xxxx: encode = 4'd13;
            16'b0001_xxxx_xxxx_xxxx: encode = 4'd12;
            16'b0000_1xxx_xxxx_xxxx: encode = 4'd11;
            16'b0000_01xx_xxxx_xxxx: encode = 4'd10;
            16'b0000_001x_xxxx_xxxx: encode = 4'd9;
            16'b0000_0001_xxxx_xxxx: encode = 4'd8;
            16'b0000_0000_1xxx_xxxx: encode = 4'd7;
            16'b0000_0000_01xx_xxxx: encode = 4'd6;
            16'b0000_0000_001x_xxxx: encode = 4'd5;
            16'b0000_0000_0001_xxxx: encode = 4'd4;
            16'b0000_0000_0000_1xxx: encode = 4'd3;
            16'b0000_0000_0000_01xx: encode = 4'd2;
            16'b0000_0000_0000_001x: encode = 4'd1;
            default: encode = 4'd0;
        endcase
    end
endfunction

// Read switches continuously (MMIO read)
reg [15:0] sw_sample;
reg [15:0] sw_prev;

wire [3:0] preview_value = encode(sw_sample);
wire any_sw_rise = |(sw_sample & ~sw_prev);

always @(posedge clk) begin
    if (rst) begin
        sw_readEnable <= 1'b0;
        sw_sample <= 16'd0;
        sw_prev <= 16'd0;
    end else begin
        sw_readEnable <= 1'b1;
        sw_prev <= sw_sample;
        sw_sample <= sw_readData[15:0];
    end
end

// -----
// FSM (same behavior as your Task01)
// -----
localparam IDLE = 2'b00;
localparam COUNT = 2'b01;
localparam DONE = 2'b10;

reg [1:0] state;
reg [3:0] counter;

always @(posedge clk) begin
    if (rst) begin
        state <= IDLE;
    end
```



```
        counter <= 4'd0;
    end else begin
        case (state)
            IDLE: begin
                counter <= preview_value; // preview

                if (any_sw_rise && (preview_value != 4'd0)) begin
                    counter <= preview_value; // latch once
                    state <= COUNT;
                end
            end

            COUNT: begin
                if (tick_1hz) begin
                    if (counter > 0)
                        counter <= counter - 1;
                    else
                        state <= DONE;
                end
            end

            DONE: begin
                counter <= 4'd0;
                if (any_sw_rise && (preview_value != 4'd0)) begin
                    counter <= preview_value;
                    state <= COUNT;
                end
            end
        endcase
    end
end

// -----
// Drive LED peripheral (MMIO write)
// same mapping as before:
// led[3:0]=counter, led[5:4]=state, led[9:6]=preview
// -----
always @(posedge clk) begin
    if (rst) begin
        led_writeEnable <= 1'b0;
        led_writeData <= 32'd0;
    end else begin
        led_writeEnable <= 1'b1;
        led_writeData <= {22'd0, preview_value, state, counter};
    end
end

// -----
// 7-seg HEX (0..F) rightmost digit
// -----
assign an = 4'b1110;

function [6:0] hex7seg;
    input [3:0] x;
    begin
        case (x)
            4'h0: hex7seg = 7'b1000000;
            4'h1: hex7seg = 7'b1111001;
            4'h2: hex7seg = 7'b0100100;
            4'h3: hex7seg = 7'b0110000;
            4'h4: hex7seg = 7'b0011001;
            4'h5: hex7seg = 7'b0010010;
            4'h6: hex7seg = 7'b0000010;
```

```

4'h7: hex7seg = 7'b1111000;
4'h8: hex7seg = 7'b0000000;
4'h9: hex7seg = 7'b0010000;
4'hA: hex7seg = 7'b0001000;
4'hB: hex7seg = 7'b0000011;
4'hC: hex7seg = 7'b1000110;
4'hD: hex7seg = 7'b0100001;
4'hE: hex7seg = 7'b0000110;
4'hF: hex7seg = 7'b0001110;
endcase
end
endfunction
assign seg = hex7seg(counter);
endmodule

```

Testbench file for FSM and counter

```

`timescale 1ns / 1ps

module Task01_using_peripherals_tb;

// -----
// Parameters - match the REAL hardware values exactly
// No parameter overrides needed; design runs as on board
// -----
localparam integer CLK_PERIOD = 10; // 10ns = 100 MHz
localparam integer DB_CYCLES = 2_000_000; // debouncer COUNT_MAX
localparam integer TICK_CYCLES = 100_000_000; // 1Hz tick DIV_MAX

// -----
// DUT signals
// -----
reg clk;
reg btnC;
reg [15:0] sw;
wire [6:0] seg;
wire [3:0] an;
wire [15:0] led;

// -----
// Instantiate DUT - no overrides, pure hardware config
// -----
Task01_using_peripherals dut (
    .clk (clk),
    .btnC(btnC),
    .sw (sw),
    .seg (seg),
    .an (an),
    .led (led)
);

// -----
// Clock generation
// -----
initial clk = 0;
always #(CLK_PERIOD/2) clk = ~clk;

// -----
// Helper task: apply reset
// Holds btnC HIGH for DB_CYCLES+100 clocks so debouncer completes
// -----
task do_reset;
begin
    $display("[RESET] Asserting btnC HIGH for %0d cycles...", DB_CYCLES+100);
    btnC = 1;
    #(CLK_PERIOD * (DB_CYCLES + 100)); // outlast the debouncer
    btnC = 0;
    #(CLK_PERIOD * 100); // settle
    $display("[RESET] Done. rst should have fired.");
end
endtask

// -----
// Helper task: press a switch value, hold briefly, then release
// -----

```



```
task press_sw;
input [15:0] sw_val;
input integer hold_cycles;
begin
    sw = sw_val;
    #(CLK_PERIOD * hold_cycles);
    sw = 16'd0;
end
endtask

// -----
// Helper: wait for N complete 1Hz ticks
// One tick = TICK_CYCLES clocks = 1 real second
// -----
task wait_ticks;
input integer n;
begin
    #(CLK_PERIOD * TICK_CYCLES * n);
end
endtask

// -----
// Test sequence
// -----
initial begin
    // Initialise
    btnC = 0;
    sw = 16'd0;
    #(CLK_PERIOD * 10);

    // =====
    // RESET - hold btnC long enough for the real debouncer
    // =====
    $display("\n=====");
    $display("[RESET] Starting reset sequence");
    $display("=====");
    do_reset;
    $display("[RESET] Expecting: state=IDLE, counter=0, seg=1000000 (digit 0)");
    #(CLK_PERIOD * 50);

    // =====
    // TEST 1: sw[2] only -> priority encode = 2
    // FSM: IDLE -> COUNT(2) -> COUNT(1) -> COUNT(0) -> DONE
    // =====
    $display("\n=====");
    $display("[Test 1] sw[2] HIGH => priority = 2");
    $display("    Expect: latch 2, count 2->1->0, then DONE");
    $display("=====");
    press_sw(16'b0000_0000_0000_0100, 10);
    wait_ticks(4); // 2 ticks to count down + 2 margin
    $display("[Test 1] Done. Expecting state=DONE, counter=0");
    #(CLK_PERIOD * 50);

    // =====
    // TEST 2: sw[15] only -> priority encode = 15
    // =====
    $display("\n=====");
    $display("[Test 2] sw[15] HIGH => priority = 15");
    $display("    Expect: latch 15, count 15->0, then DONE");
    $display("=====");
    press_sw(16'b1000_0000_0000_0000, 10);
    wait_ticks(17); // 15 ticks + 2 margin
    $display("[Test 2] Done. Expecting state=DONE, counter=0");
    #(CLK_PERIOD * 50);

    // =====
    // TEST 3: sw[14] and sw[2] both HIGH -> priority = 14
    // =====
    $display("\n=====");
    $display("[Test 3] sw[14]+sw[2] => priority = 14 (highest wins)");
    $display("    Expect: latch 14, count 14->0, then DONE");
    $display("=====");
    press_sw(16'b0100_0000_0000_0100, 10);
    wait_ticks(16); // 14 ticks + 2 margin
    $display("[Test 3] Done. Expecting state=DONE, counter=0");
    #(CLK_PERIOD * 50);

    // =====
    // TEST 4: Re-trigger from DONE
    // =====
```

```
// =====
$display("\n=====");
$display("[Test 4] Re-trigger from DONE state");
$display("    Step A: sw[1] => priority 1, wait for DONE");
$display("=====");
press_sw(16'b0000_0000_0000_0010, 10); // sw[1] = priority 1
wait_ticks(3);
$display("[Test 4] In DONE. Now pressing sw[3] to re-trigger...");
press_sw(16'b0000_0000_0000_1000, 10); // sw[3] = priority 3
wait_ticks(5);
$display("[Test 4] Done. Expecting state=DONE, counter=0");
#(CLK_PERIOD * 50);

// =====
// TEST 5: No switch -> stays IDLE
// =====
$display("\n=====");
$display("[Test 5] No switch pressed -> stays IDLE");
$display("=====");
wait_ticks(3);
$display("[Test 5] Done. Expecting state=IDLE, counter=0");

// =====
// TEST 6: Reset mid-count
// =====
$display("\n=====");
$display("[Test 6] Reset during COUNT");
$display("    sw[8] => priority 8, reset after 3 ticks");
$display("=====");
press_sw(16'b0000_0001_0000_0000, 10); // sw[8] = priority 8
wait_ticks(3);
$display("[Test 6] Applying reset now (counter should be ~5)...");
do_reset;
$display("[Test 6] Done. Expecting state=IDLE, counter=0");
#(CLK_PERIOD * 50);

$display("\n=====");
$display("===== All Tests Complete =====");
$display("=====");
$finish;
end

// -----
// Decode LED fields
// led[3:0] = counter
// led[5:4] = state (00=IDLE, 01=COUNT, 10=DONE)
// led[9:6] = preview_value
// -----
reg [1:0]  state_bits;
reg [3:0]  counter_bits;
reg [3:0]  preview_bits;
reg [8*5-1:0] state_name;

always @(*) begin
    counter_bits = led[3:0];
    state_bits  = led[5:4];
    preview_bits = led[9:6];
    case (state_bits)
        2'b00: state_name = "IDLE ";
        2'b01: state_name = "COUNT";
        2'b10: state_name = "DONE ";
        default: state_name = "?????";
    endcase
end

// -----
// Print only on changes to avoid billions of identical lines
// -----
reg [1:0] last_state;
reg [3:0] last_counter;
reg [3:0] last_preview;
reg      last_btnC;
reg [15:0] last_sw;

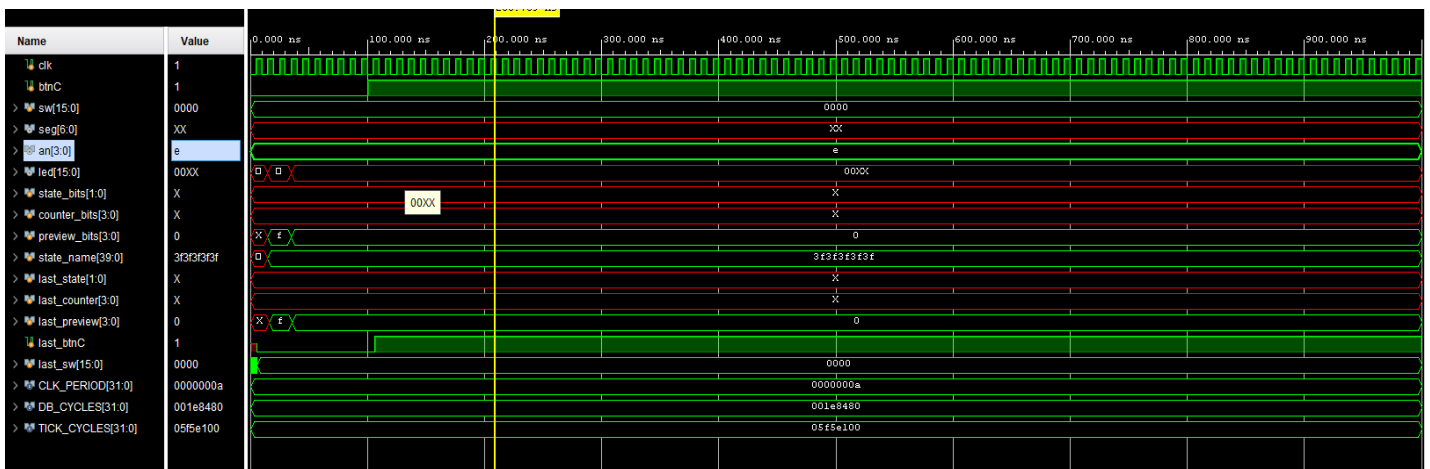
initial begin
    last_state = 2'bxx;
    last_counter = 4'bxxxx;
    last_preview = 4'bxxxx;
    last_btnC = 1'bx;
end
```

```

last_sw      = 16'bx;
$display("\n%-14s | %-5s | %-5s | %-16s | %-5s | ctr | prev | seg",
        "Time(ns)", "btnC", "rst", "sw", "state");
end

always @(posedge clk) begin
    #1;
    if (state_bits !== last_state ||
        counter_bits !== last_counter ||
        preview_bits !== last_preview ||
        btnC      !== last_btnC ||
        sw        !== last_sw)
    begin
        $display("%-14t | %-5b | %-5b | %b | %s | %2d | %2d | %7b",
            $time,
            btnC,
            dut.rst,
            sw,
            state_name,
            counter_bits,
            preview_bits,
            seg);
        last_state <= state_bits;
        last_counter <= counter_bits;
        last_preview <= preview_bits;
        last_btnC <= btnC;
        last_sw <= sw;
    end
end
end
```

Simulation waveforms showing correct operation



Task 3: FPGA Constraints, XDC File and Timing Analysis

XDC file

```
set_property PACKAGE_PIN E3 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -period 10.000 -name sys_clk [get_ports clk]

## Reset (btnC center button)
set_property PACKAGE_PIN N17 [get_ports btnC]
set_property IOSTANDARD LVCMOS33 [get_ports btnC]

## Switches
set_property PACKAGE_PIN J15 [get_ports {sw[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
set_property PACKAGE_PIN L16 [get_ports {sw[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
set_property PACKAGE_PIN M13 [get_ports {sw[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
set_property PACKAGE_PIN R15 [get_ports {sw[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
set_property PACKAGE_PIN R17 [get_ports {sw[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
set_property PACKAGE_PIN T18 [get_ports {sw[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
set_property PACKAGE_PIN U18 [get_ports {sw[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
set_property PACKAGE_PIN R13 [get_ports {sw[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
set_property PACKAGE_PIN T8 [get_ports {sw[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[8]}]
set_property PACKAGE_PIN U8 [get_ports {sw[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[9]}]
set_property PACKAGE_PIN R16 [get_ports {sw[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[10]}]
set_property PACKAGE_PIN T13 [get_ports {sw[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[11]}]
set_property PACKAGE_PIN H6 [get_ports {sw[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[12]}]
set_property PACKAGE_PIN U12 [get_ports {sw[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[13]}]
set_property PACKAGE_PIN U11 [get_ports {sw[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[14]}]
set_property PACKAGE_PIN V10 [get_ports {sw[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[15]}]

## LEDs
set_property PACKAGE_PIN H17 [get_ports {led[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property PACKAGE_PIN K15 [get_ports {led[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
set_property PACKAGE_PIN J13 [get_ports {led[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property PACKAGE_PIN N14 [get_ports {led[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
set_property PACKAGE_PIN R18 [get_ports {led[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
set_property PACKAGE_PIN V17 [get_ports {led[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
set_property PACKAGE_PIN U17 [get_ports {led[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
set_property PACKAGE_PIN U16 [get_ports {led[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
set_property PACKAGE_PIN V16 [get_ports {led[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[8]}]
set_property PACKAGE_PIN T15 [get_ports {led[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[9]}]
set_property PACKAGE_PIN U14 [get_ports {led[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[10]}]
set_property PACKAGE_PIN T16 [get_ports {led[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[11]}]
set_property PACKAGE_PIN V15 [get_ports {led[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[12]}]
set_property PACKAGE_PIN V14 [get_ports {led[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[13]}]
set_property PACKAGE_PIN V12 [get_ports {led[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[14]}]
```



```
set_property PACKAGE_PIN V11 [get_ports {led[15]]}
set_property IOSTANDARD LVCMOS33 [get_ports {led[15]]}

## 7-segment display
set_property PACKAGE_PIN T10 [get_ports {seg[0]]}
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]]}
set_property PACKAGE_PIN R10 [get_ports {seg[1]]}
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]]}
set_property PACKAGE_PIN K16 [get_ports {seg[2]]}
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]]}
set_property PACKAGE_PIN K13 [get_ports {seg[3]]}
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]]}
set_property PACKAGE_PIN P15 [get_ports {seg[4]]}
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]]}
set_property PACKAGE_PIN T11 [get_ports {seg[5]]}
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]]}
set_property PACKAGE_PIN L18 [get_ports {seg[6]]}
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]]}

## 7-segment anodes
set_property PACKAGE_PIN J17 [get_ports {an[0]]}
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]]}
set_property PACKAGE_PIN J18 [get_ports {an[1]]}
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]]}
set_property PACKAGE_PIN T9 [get_ports {an[2]]}
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]]}
set_property PACKAGE_PIN J14 [get_ports {an[3]]}
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]]}
```

Screenshot of Vivado Timing Summary

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4.595 ns	Worst Hold Slack (WHS): 0.145 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 286	Total Number of Endpoints: 286	Total Number of Endpoints: 146

All user specified timing constraints are met.

Clock Summary

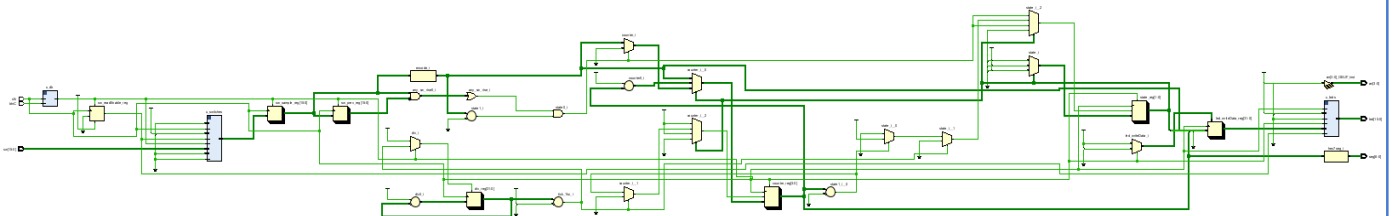
Name	Waveform	Period (ns)	Frequency (MHz)
sys_clk	{0.000 5.000}	10.000	100.000

WNS value and interpretation (PASS / MARGINAL / FAIL)

Tcl Console		Messages	Log	Reports	Design Runs	Power	DRC	Methodology	Timing						?	_	□
Q⌵⌶⏮⏪⏩⏭+%																	
Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAM	URAM				
✓ synth_1	constrs_1	synth_design Complete!								89	145	0.0	0				
✓ impl_1	constrs_1	route_design Complete!	4.595	0.000	0.145	0.000	0.000	0.102	0	88	145	0.0	0				

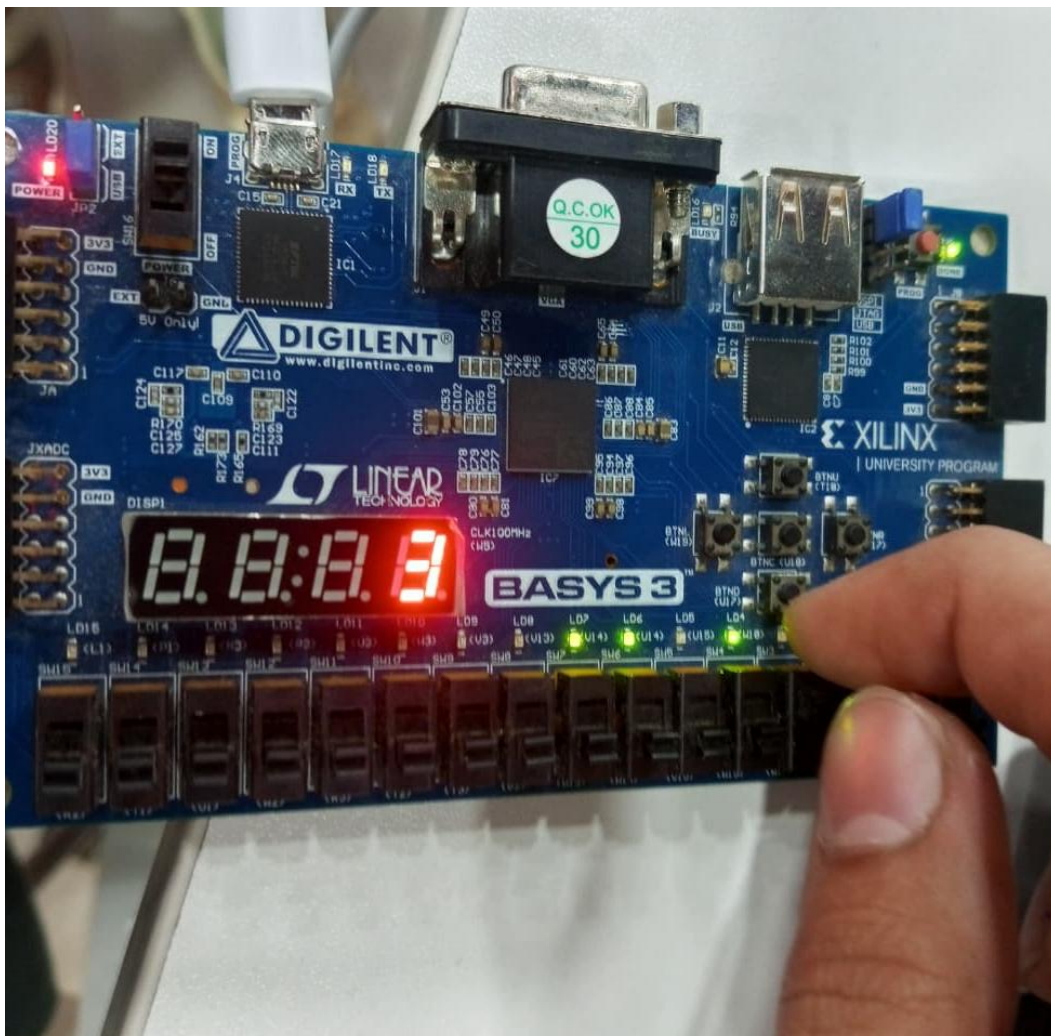
Task 4: FPGA Synthesis and On-Board Verification

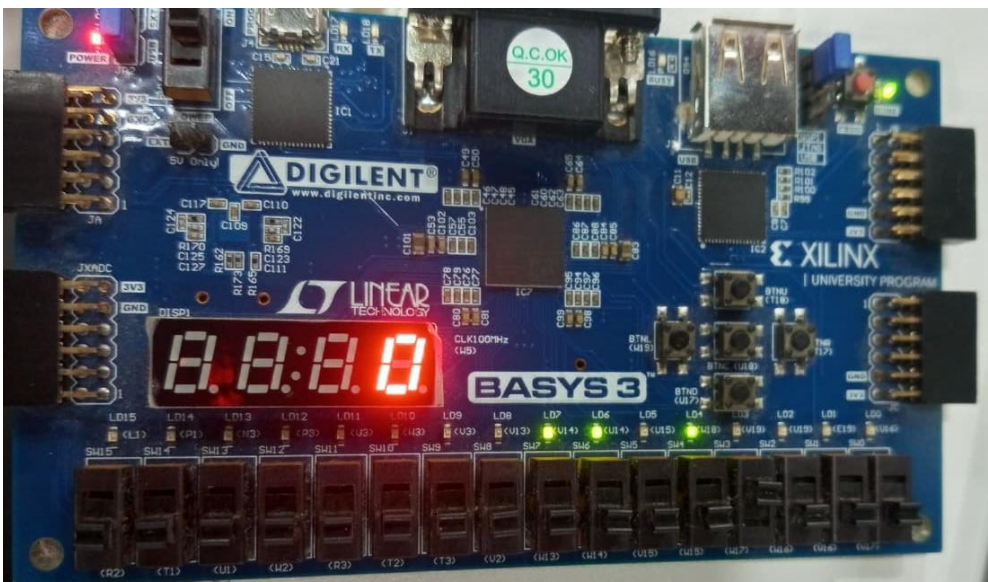
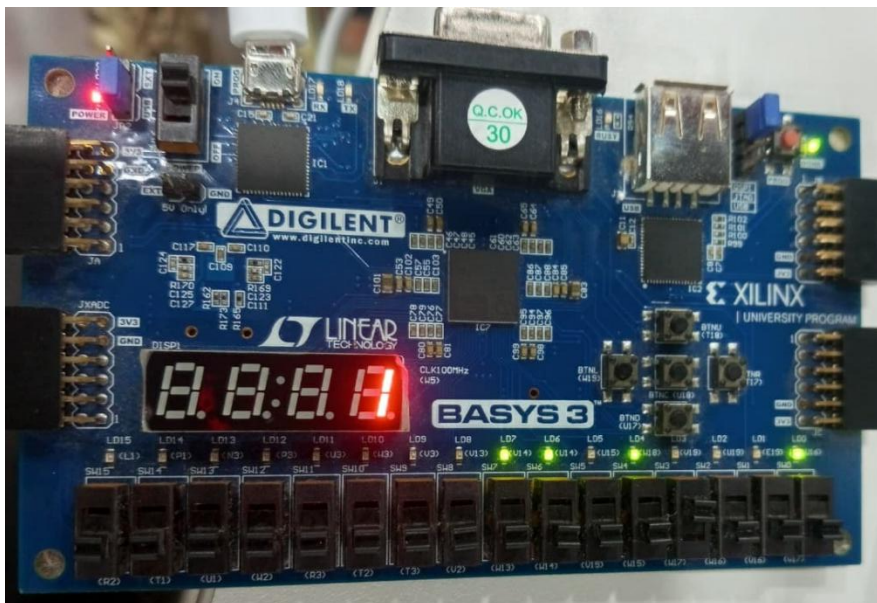
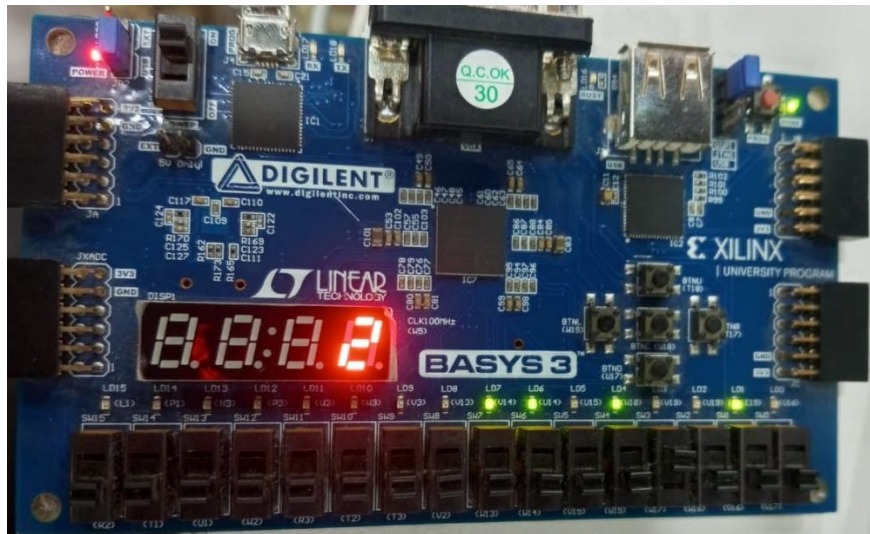
RTL schematic screenshots



Successful FPGA demonstration

COUNTDOWN FROM 3 TO ZERO (Because we turned on 3rd Switch)







Assessment Rubric

Lab 5: Designing FSM Using FPGA Switches and LEDs

Name:	Student ID:	Section:
-------	-------------	----------

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Pre lab	-	/15	-
	Task 1	-	-	/10
	Task 2	/15	/15	-
	Task 3	-	/10	-
	Task 4	-	/15	
Total Points		/15	/55	/10
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.

